Vibe Fighter — Prompt Recipe

The key prompts behind the build: Codex ($imagegen) for the art, Cursor for the game.

3. Architect the Fighting Game (Cursor)

Cursor · build

Plan a minimal-but-authentic fighting-game architecture on top of the starter — reuse the per-frame boxes, don't rewrite.

```
Let's consider what we have here as a template. We now want to build a 2D fighting game like
Street Fighter on top of this starting project. There already exists several scenes and
functionality that will help with debugging, testing, and getting the main game in.

Consider an architecture for a fighting game that is representative of what is done in a
fighting game, but not overly complex. The key here is the animation + collision + gameplay
logic. A traditional fighter has many animation states (forward, backward, block high, block
low, attack soft/medium/hard, etc.) — but let's focus on the basic animations required for a
compelling first pass and decide where to build on later.

Before writing code, audit what the starter already gives us and tell me what is missing for a
fighter. Then write a detailed implementation plan.
```

```
Constraints for the first pass: two local human players sharing one keyboard (P1 + P2); two
attack buttons — light and heavy; a new dedicated match scene; write the plan first, then build
incrementally.
```

---

4. Generate the Concept Mockups (Codex $imagegen)

Codex · $imagegen

Create the visual target for the whole game: four SNES-style mockups that read as a Street Fighter type game, then lock a direction.

```
$imagegen — create 4 concept mockups for this 2D fighting game. Place the art in
concepts/mockups/<timestamp>/ and save the exact prompts you used alongside them.

Focus on art style and overall feel — nothing too complex, this is a technical demo. They should
be easily recognisable as a Street Fighter type game, leaning SNES pixel art rather than high-
resolution comic style. 4 different versions.
```

---

5. Extract the Parallax Backgrounds (Codex $imagegen)

Codex · $imagegen

Split the chosen stage into layered, transparent parallax PNGs the game can scroll independently.

```
The rooftop-dusk background is the one. Split it into separate PNGs with transparency so they
can be layered on each other: a far (sky), a medium (buildings), a main (the roof / main
fighting area), and a near layer (drawn in front of the main area and the fighters). Use
$imagegen to create each one individually and place them in concepts/backgrounds/rooftop-dusk/.
```

```
The main rooftop layer is not end-to-end to the edge of the canvas, so it will tile with gaps
and look weird. Regenerate it full-width.
```

---

6. Generate the Character References (Codex $imagegen)

Codex · $imagegen

Pull a clean, full-body fighter reference out of each mockup — the source art the bundled animation spritesheets were made from.

```
Create a clean full-body reference image for an original SNES-era pixel-art fighting-game character inspired by the left fighter in the rooftop mockup: an athletic young street brawler, spiky brown hair, red cropped jacket over a light shirt, dark pants, red sneakers, fingerless gloves, confident martial-arts stance. Isolated full-body, 3/4 side-facing fighting stance, centered with generous padding, no stage, no HUD, no opponent. Original character only, no logos, no text. Save to concepts/characters/<timestamp>/ with its prompt.
```

```
Using sub-agents, generate two more character references with $imagegen: the green boxer from the dojo mockup, and the jiu-jitsu fighter on the right from the subway mockup. Same SNES style, isolated full-body. Save them under concepts/characters/<timestamp>/ with both the draft and final prompts.
```

---

7. Generate the Character-Select Portraits (Codex $imagegen)

Codex · $imagegen

Create a stylised portrait for each fighter for the big character-select cards.

```
Generate a stylised portrait of each character based on their reference images. Frame the upper body to head in distinctive poses suited for characters — these will be used in a character-selection screen. $imagegen. Keep the SNES pixel-art style, just higher fidelity for a portrait while maintaining the vibe.
```

---

8. Generate the UI + Prop Atlases (Codex $imagegen)

Codex · $imagegen

Create the HUD art (health bar with a transparent fill slot, portrait base) and a set of animated stage props.

```
Generate a UI atlas with $imagegen we can use for the game — a health bar (with appropriate transparent areas for the dynamic health fill) and a portrait base we can drop each character's portrait into. Use a magenta (#ff00ff) chroma background for the transparent areas so we can key it out locally.

In parallel, generate a separate atlas of animated props for the twilight stage — crowds, roof vents, beacons, steam — that we can use to give the background some depth. $imagegen.
```

---

9. Register Fighters & Author Per-Frame Hitboxes (Cursor)

Cursor · build

Wire the three fighters into the Character Gym and author the per-frame hit / hurt / attack boxes — including each attack's active-frame window.

```
Integrate the bundled fighter assets into the Character Gym. Each fighter should be one data entry (id, asset paths, animation keys, fps, frame size, anchor) so it auto-wires preload, the animation registry, and the gym's dropdown. Do not auto-fit bounds from sprite alpha — leave placeholder boxes; I'll author them by hand and save to public/configs/character-gym.json.
```

```
How hard is it to adjust the bounds in the Character Gym via gizmos like in Unreal / Unity / Blender — two-axis arrows and free movement? Add a mode switcher in the debug panel, hotkeyed to Q (translate) and W (scale). Grabbing a box should auto-pause on the current frame and write to character-gym.json, flowing to the sidebar fields and live combat.
```

---

10. Build the Fighter Playground (Cursor)

Cursor · build

A sandbox to test how a fighter feels — move and act on a fixed axis, tune per-character stats live, and visualise the debug bounds.

```
Add a Playground scene: take the existing fighter and the rooftop-twilight stage and let the player move left and right on a fixed axis and perform all their actions.
```

```
Make the fighters selectable in the playground and the Character Gym. In the playground, expose per-character stats in the debug panel that can be persisted to a config — walk speed, jump, gravity, scale — so we can balance each fighter and have it apply in the main game.
```

```
While typing in the debug panel, input must not accidentally move or affect the player in the game. And after I tweak a value and click on the canvas, keyboard control of the game must come back.
```

```
Add toggles to visualise all the debug bounds — hit box, attack box, collision — each in a different colour, easily toggleable with a toggle-all. For bounds that are only active on certain frames, render them faint when inactive but still perceivable, and solid when active.
```

---

11. Menus, Portraits & the Versus Flow (Cursor)

Cursor · build

Thread the Play flow — mode, stage, character select — into a versus match, using the portraits on big cards.

```
Now let's get the main flow in. Play -> 1v1 or 1vCPU. Level selection -> [rooftop twilight, rooftop sunset] using nice cards that show the names. In character selection, if it's 1vCPU the player picks freely and then the CPU picks; add a nice selection effect — a brief pause while the choice flashes to lock in. In 1v1, WASD moves the selection for P1 (indicated) and the arrows move P2 (indicated); they can't choose the same character, so they skip over one. Make the character-select screen nice and use the portraits — big cards. Start the match with a 'Round 1... Fight!' text before the players can move and attack.
```

```
In the playground we need to be able to save the character stats to file as a config so they apply in the main game and can be loaded. (When I tried to save the jiu-jitsu fighter's scale it said 'target not found'.) For now restrict the selectable characters to the brawler and the jiu-jitsu fighter; in 1v1 moving the selection just swaps them since there's no room. Also show the bounds toggle in the main match too.
```

---

12. Core Combat: Hits, Rounds, Timer & Camera (Cursor)

Cursor · build

Make hits land from the authored boxes, add best-of-3 rounds, a round timer, a group camera, and correct z-order and facing.

```
How do we deal with z-ordering of the sprites? Right now P1 is perpetually behind P2. Is it whoever moves most recently is drawn in front? And when the players are on the other side of each other, their sprites + colliders should flip to face the other way.
```

```
Now let's implement the core of the game: taking damage, hit detection, knockdown. We should have some configurability per character — e.g. attack low and attack high can differ — written to config and adjustable. Let's have 3 rounds before the match goes game over, then it's rematch or main menu.
```

```
How do Street-Fighter-like games deal with scrolling on stages larger than the viewport — the camera stays locked until one of them gives way, right? Add that. And add a round timer.
```

---

13. Geometric Guard-Box Blocking (Cursor)

Cursor · build

Add blocking as a new bounds kind — a guard box — so high/low coverage emerges from geometry, not from labels.

```
Let's model blocking with geometry, not height labels. Give block animations a `guard` box. A hit is BLOCKED when the attacker's hitbox overlaps the defender's guard box; high vs low is just where the boxes sit. Resolve a hit as two independent checks: (1) geometry — did the hitbox overlap the hurtbox at all, else it whiffs; and (2) guard — was the right stance up, else it connects. And pipe the gym's saved boxes into live combat so tuning a box in the gym changes the match.
```

---

14. Build the HUD (Cursor)

Cursor · build

Lay out both fighters' health bars and portraits from the UI atlas, with dynamic colour, a low-health blink, and a match-start entrance.

```
Integrate the UI elements into the game. Lay out health for the two characters and their portraits — make it look good. The health bar should be dynamic and change colour based on the amount of health left, plus blink when low. Add some dynamism: at match start the health bars fill and the UI slides down.
```

```
The slide-down of the UI and the fill need to be a bit slower — right now it's too rapid.
```

---

15. Meter Specials & Multi-Hit Combos (Cursor)

Cursor · build

Add a meter-gated special system with multi-hit combos and a finisher — proven on a training dummy.

```
I'd like a 'special moves' system — hit a specials button when a meter is filled. Define the specials for the brawler and the jiu-jitsu fighter; they each need a charging animation and an execution animation (the jiu-jitsu guy could do a 'spinarooni', the brawler a revolving uppercut). The key thing: how do we register multiple hits across an animation? Right now we register one hit per attack animation, but specials usually have a combo. Also consider what on-screen effect is needed. (The special charge/execution spritesheets are already bundled.)
```

```
The playground needs a dummy character and a debug toggle to fill the player's special bar. The dummy should have a health bar that constantly regenerates after hits so we can visualise the impact of attacks. I just tried the jiu-jitsu special on the dummy — how many hits do we expect?
```
